

Pitch técnico

Buenos días. Somos el equipo desarrollador de **TurQuiYA**, una Progressive Web Application creada para promover el turismo en Barranquilla y el departamento del Atlántico.

Identificamos que gran parte de la información turística se encuentra dispersa en diferentes plataformas, lo que dificulta que residentes y visitantes descubran fácilmente los atractivos de la región. Además, los turistas no conocen los precios reales de transporte, la cobertura móvil en zonas rurales ni cuánto dinero necesitan para su estancia.

Nuestro objetivo fue desarrollar una aplicación centralizada, accesible desde cualquier dispositivo, que permitiera consultar información turística de manera sencilla, sin necesidad de instalar una app desde una tienda. Para lograrlo diseñamos una solución basada en tecnologías web modernas y servicios en la nube, priorizando la facilidad de mantenimiento, el bajo costo de despliegue y la escalabilidad del proyecto.

La aplicación ya está desplegada y funcional en turquiya.dpdns.org, permitiendo a cualquier usuario registrarse e iniciar sesión desde cualquier dispositivo con su correo y contraseña.

A continuación explicaremos las decisiones técnicas que tomamos durante el desarrollo

La arquitectura de TurQuiYA está basada en una separación clara entre el cliente y los servicios en la nube. Seguimos un modelo **cliente-servidor desacoplado**, donde el frontend consume servicios backend mediante APIs REST.

¿Por qué HTML, CSS y JavaScript Vanilla? Elegimos JavaScript Vanilla en lugar de React, Vue o Angular porque nuestro objetivo pedagógico era comprender a profundidad el funcionamiento del DOM, el consumo de APIs y la organización modular del código antes de utilizar un framework. Para el tamaño del proyecto, JavaScript Vanilla resultó suficiente y evitó incorporar dependencias innecesarias que aumentarían el tiempo de carga. Organizamos el código en módulos ES6 con import/export, lo que nos permite mantener el código limpio y escalable sin necesidad de bundlers como Webpack.

¿Por qué Progressive Web Application (PWA)? Optamos por una PWA en lugar de una aplicación nativa porque nos permite mantener una única base de código para todos los dispositivos, facilitar el despliegue, reducir costos de

desarrollo y ofrecer una experiencia cercana a una app nativa con instalación directa desde el navegador.

Modelo de Datos: Diseñamos el modelo de datos en PostgreSQL con tres tablas principales: perfiles para los usuarios registrados, destinos para los lugares turísticos, y categorías para clasificarlos. Usamos claves foráneas para relacionar destinos con categorías, y el tipo **JSONB** para datos semiestructurados como las tarifas de transporte y la cobertura móvil por operador. Implementamos **Row Level Security (RLS)** para que cada usuario solo acceda a su propia información, mientras que los administradores pueden gestionar todo el contenido.

¿Por qué Supabase como Backend? Para el backend decidimos utilizar **Supabase** como Backend as a Service. En lugar de desarrollar una API propia con Node.js y Express, consumimos directamente la API REST que Supabase genera automáticamente sobre nuestra base de datos PostgreSQL.

Esta decisión redujo significativamente el tiempo de desarrollo y nos permitió concentrarnos en la lógica de negocio y la experiencia del usuario. Elegimos Supabase en lugar de Firebase porque utiliza **PostgreSQL**, una base de datos relacional que facilita el manejo de relaciones entre entidades mediante SQL, ofrece autenticación, almacenamiento y una API REST automática dentro de una misma plataforma, y su plan gratuito es más generoso para proyectos en etapa de MVP.

Despliegue: El frontend se encuentra desplegado en **GitHub Pages** y los servicios backend en **Supabase Cloud**, lo que nos da un despliegue completamente serverless sin necesidad de administrar servidores propios.

Organización del código con GitFlow: Organizamos el desarrollo con **GitFlow**: la rama main para código en producción, develop para integración continua, y ramas feature/nombre-de-la-hu por cada historia de usuario. Usamos **GitHub Projects** con tablero Kanban para visualizar el progreso de los sprints, asignar tareas y hacer seguimiento diario mediante standups en Discord. La convención de commits sigue el formato feat:, fix:, docs:, style: y refactor: para mantener un historial claro.

Autenticación y Seguridad: La autenticación la manejamos con **Supabase Auth**, que utiliza **JWT (JSON Web Tokens)** con refresh tokens automáticos. Cuando el usuario inicia sesión, Supabase genera un token de acceso y otro de refresco. El token se almacena en localStorage para mantener la sesión activa

incluso si cierra el navegador. Implementamos `supabase.auth.getSession()` para verificar la sesión activa al cargar la aplicación, evitando depender solo de `localStorage` y garantizando que el usuario pueda acceder desde cualquier dispositivo.

Configuramos la persistencia de sesión con `autoRefreshToken: true` y `persistSession: true`, lo que permite que el token se renueve automáticamente antes de expirar.

Funcionalidades implementadas:

- Navegación dinámica SPA con router personalizado en JavaScript vanilla
- Registro e inicio de sesión con Supabase Auth, JWT y sesiones persistentes
- **Planificador de viajes por presupuesto:** el usuario ingresa su presupuesto en COP y el sistema recomienda destinos que se ajusten
- Consulta de destinos turísticos con filtros por categoría
- Calculadora de cambio de moneda multi-divisa (USD, EUR, ARS, BRL, CLP)
- Consumo de la API REST de Supabase mediante `@supabase/supabase-js` vía CDN
- Menú dinámico que se adapta según el estado de autenticación del usuario
- Configuración mediante variables de entorno para desacoplar credenciales del código
- Capacidades PWA: instalación en móviles, diseño responsive, iconos y manifest

Durante el desarrollo enfrentamos retos técnicos específicos. La configuración de **Row Level Security (RLS)** en Supabase nos tomó varias iteraciones porque inicialmente los usuarios no podían leer sus propios perfiles. Lo resolvimos creando políticas RLS que verifican el `auth.uid()` contra el `id` de la tabla `perfiles`.

También tuvimos que resolver problemas de **CORS** al consumir la API desde el frontend desplegado en GitHub Pages, configurando correctamente los headers en Supabase. Para el manejo de sesiones, implementamos verificación dual: primero consultamos `supabase.auth.getSession()` y luego validamos los datos en `localStorage`, asegurando que la sesión sea válida en el servidor y no solo en el navegador.

Como trabajo futuro identificamos:

- Recomendaciones personalizadas para los usuarios basadas en historial
- Búsqueda avanzada y filtros por múltiples criterios
- Integración con servicios de geolocalización en tiempo real
- Sistema de favoritos y listas guardadas
- Comentarios y calificaciones de destinos
- Notificaciones push
- Panel administrativo más completo con analítica de uso

La arquitectura actual facilita la incorporación de estas funcionalidades sin necesidad de rediseñar completamente el sistema.

En conclusión, TurQuiYA demuestra cómo es posible construir una plataforma moderna utilizando tecnologías web y servicios en la nube. Las decisiones técnicas que tomamos buscaron equilibrar simplicidad, mantenibilidad y escalabilidad, permitiéndonos desarrollar una solución funcional sin administrar infraestructura propia.

Consideramos que la combinación de **JavaScript Vanilla, Supabase, PostgreSQL, GitHub Pages y el enfoque de Progressive Web Application** constituye una base sólida para continuar evolucionando el proyecto.

Muchas gracias por su atención. Quedamos atentos a sus preguntas.